

3. Listas, Dicionários, Tuplas e Funções.

▼ Lista (coleção ou arrays)

▼ Introdução às Listas

- É uma coleção **ordenada** e **mutável** de elementos.
- Pode conter elementos de diferentes tipos (inteiros, strings, objetos, etc.).

```
#sintaxe  
minha_lista = [1, 2, 3, 4, 5]
```

- Permitem duplicatas.
- Ordenadas (a ordem de inserção é mantida).
- Indexadas (cada elemento tem um índice).

▼ Principais Operações com Listas

▼ Acessando elementos:

```
minha_lista[0]  # Primeiro elemento  
minha_lista[-1] # Último elemento
```

▼ Adicionando elementos:

- **append** - Adiciona ao final.
- **insert** - Adiciona em uma posição específica.

```
minha_lista.append(6)  
minha_lista.insert(2, "novo_item")
```

▼ Removendo elementos:

- **remove** : Remove por valor.
- **pop** : Remove por índice (e retorna o valor).

```
minha_lista.remove(3)
item = minha_lista.pop(0)
```

▼ Outras operações úteis:

- `len`: Retorna o tamanho da lista.
- `sort`: Ordena os elementos.
- `reverse`: Inverte a ordem.

```
minha_lista.sort()
minha_lista.reverse()
```

▼ Exemplos Práticos com Listas

▼ Exemplo 1: Adicionar produtos ao carrinho

```
carrinho = []
carrinho.append("produto_1")
carrinho.append("produto_2")
print(carrinho) # ['produto_1', 'produto_2']
```

▼ Exemplo 2: Filtro de dados de uma API (simulação)

```
usuarios = ["Alice", "Bob", "Carlos"]
usuarios_filtrados = [u for u in usuarios if "a" in u.lower()]
print(usuarios_filtrados) # ['Alice', 'Carlos']
```

▼ Métodos de lista

Como vimos, o Python permite manipular listas de várias formas. Resumimos com uma lista completa de métodos da classe *list*.

Método	Descrição
<code>list.append(x)</code>	Adiciona um novo elemento ao final da lista.

<code>list1.extend(list2)</code>	Adiciona um grupo de elementos (iterables) ao final da lista ou concatena listas.
<code>list.insert(índice, elemento)</code>	Insere um elemento em uma posição específica da lista.
<code>list.remove(elemento)</code>	Exclui a primeira ocorrência de um elemento da lista.
<code>list.pop(índice)</code>	Extraí e exclui um elemento da lista na posição indicada.
<code>list.clear()</code>	Apaga todos os elementos da lista.
<code>list.index(elemento)</code>	Devolve o índice da primeira ocorrência de um elemento na lista.
<code>list.count(elemento)</code>	Conta o número de ocorrências de um elemento na lista.
<code>list.sort()</code>	Organiza os elementos da lista.
<code>list.reverse()</code>	Coloca elementos da lista em ordem inversa.
<code>list.copy()</code>	Devolve uma cópia da lista.

[ListaDeExercicios - PythonBrasil](#)

[Exercícios - Pense em Python \(caravela.club\)](#)

▼ Dicionários

▼ Introdução às Dicionários

- Um dicionário é uma coleção **desordenada**, **mutável** e **indexada por chaves**.
- Cada chave está associada a um valor.

```
#sintaxe
meu_dict = {"chave1": "valor1", "chave2": "valor2"}
```

- As chaves são únicas e imutáveis.
- Os valores podem ser de qualquer tipo.

▼ Principais Operações com Dicionários

▼ Acessando valores:

```
meu_dict["chave1"] # valor1
```

▼ Adicionando ou atualizando valores:

```
meu_dict["nova_chave"] = "novo_valor"
```

▼ Removendo itens:

- `pop`: Remove por chave.
- `clear`: Remove todos os itens.

```
meu_dict.pop("chave1")  
meu_dict.clear()
```

▼ Iterando em dicionários:

- **Chaves:** `for key in meu_dict:`
- **Valores:** `for value in meu_dict.values():`
- **Pares chave-valor:** `for key, value in meu_dict.items():`

▼ Exemplos Práticos com Dicionários

Exemplo 1: Configurações de um aplicativo web

```
config = {  
    "DEBUG": True,  
    "DATABASE": "sqlite3",  
    "API_KEY": "12345"  
}  
print(config["DATABASE"]) # sqlite3
```

Exemplo 2: Transformando dados de um formulário em JSON

```
form_data = {"nome": "João", "email": "joao@example.com"}  
print(form_data)  # {'nome': 'João', 'email': 'joao@example.com'}
```

```
livro1={  
    'titulo':"Fundamento de Python",  
    'autor':"Fulano de Tal",  
    'edicao':"2022",  
    'qtd_pagina':215  
}  
  
livro2={  
    'titulo':"Guia Pratico",  
    'autor':"Ciclano",  
    'edicao':"2015",  
    'qtd_pagina':322  
}
```

▼ Listas Vs Dicionários

Quando temos muitas informações para guardar, precisamos organizá-las. Em Python, usamos listas e dicionários. Uma lista é como uma sacola onde você pode colocar várias coisas, como uma lista de cores. Já um dicionário é como um caderno onde você anota informações, como o nome e idade de uma pessoa.

```
# Lista
cores = ['vermelho', 'verde', 'azul']
```

```
# Dicionário
pessoa = {
    'nome': 'João',
    'idade': 30,
    'cidade': 'São Paulo'
}
```

Slide 9: Comparação entre Listas e Dicionários

Aspecto	Listas	Dicionários
Estrutura	Coleção de elementos ordenados	Coleção de pares chave-valor
Indexação	Por posição (0, 1, 2...)	Por chave
Permite duplicatas?	Sim	Não (apenas para chaves)
Exemplo de uso	Carrinho de compras	Configurações de sistema

Slide 10: Exercício Prático

Exercício 1: Gerenciando dados de usuários

Crie um programa que:

1. Armazene uma lista de usuários (nome e email) em um dicionário.
2. Permita adicionar, remover e atualizar dados dos usuários.
3. Exiba os usuários registrados.

▼ Dica:

Use listas dentro de dicionários!

Slide 11: Conclusão

- **Listas e dicionários** são essenciais em Python.
- São amplamente usadas em projetos web para manipulação de dados dinâmicos.
- Pratique com exemplos reais (carrinhos de compras, respostas de APIs).

Próximos passos:

1. Explorar manipulação avançada com `json`.
 2. Aplicar essas estruturas no consumo de APIs e frameworks como Django/Flask.
-

▼ Funções

Imagine que você tem uma superpoderosa máquina que faz um trabalho para você sempre que você a chama. Em Python, essas "máquinas" são chamadas de funções. Você pode criar sua própria função para fazer algo específico sempre que precisar, como dizer "olá" para alguém.

▼ Função

- Chamando a função

```
saudacao("Ana")
```

O que é uma função

é uma função é um bloco de código reutilizável que executa um tarefa específica. você pode chama-la quantas vezes quiser, o que ajuda a evitar repetição de código. Uma função pode receber parâmetros (entradas) e retorna um valor.

▼ Definindo uma função

```
def saudacao(nome):  
    print(f'Olá {nome}, seja bem-vindo!')
```

▼ Paramentos e retorno

função podem receber valores (paramentos) e devolvem um resultado (retorno). Isso permite criar funções mais dinâmias e flexíveis.